

Lab 4: VibeCoding - Chat History

Project: 47_target-aim-trainer (Pygame) | Student: Louis Alan | Date: 8 Oct 2026 | Tool: Claude

Full chat link: <https://claude.ai/share/18b9a170-cccb-4250-9860-9a43d2686912>

This document is a condensed transcript of the Claude session: my requests in order, a summary of Claude's replies, and the code changes for each task (as git diffs). The linked page has the complete conversation.

Setup: reading the handout and the code

You: help to finish this deliverables [attached: Lab 4 VibeCoding student handout PDF]

Claude: Read the handout (fix the broken code, add the README features one by one, separate commits per task, before/after videos, chat export). Inspected the cloned repo: `main.py`, `game/game_engine.py`, `game/target.py` and the README, which lists four tasks: (1) fix collision detection, (2) game over screen, (3) replay with difficulty, (4) sound effects. Built and tested each task in a working copy, then applied them to the repo as separate commits.

Prompt 0: before video

You: i have recorded the before video now proceed with task one

Claude: Warned that the edited code might already have been in the folder when the video was recorded, and gave the commands to re-check out the original starter commit so the bug shows.

You: revert the changes back before task 1, I gotta record video with bugs

Claude: Created a branch at the original starter commit (a08501a), keeping the finished work safe on main. Bug to show: a click well outside the shrunken circle still scores a hit.

Task 1: Refine collision detection

You: now do task 1 and commit

Claude: Root cause: `Target.contains_point()` compared the click distance to `base_radius` (the starting size), while `render()` draws `visual_radius()`, which shrinks as the target ages. So the clickable area never shrank. **Fix:** hit-test against `visual_radius()`. Tested: with the target nearly fully shrunk (radius about 12 px), a click 30 px from the center no longer scores; a center click still does. Commit 9eb8ee3.

```
@@ -23,9 +23,5 @@ class Target:
    def contains_point(self, x, y):
-        # NOTE: hit-testing always uses the target's starting radius
-        # (base_radius), not its current, shrunken on-screen radius
-        # (visual_radius()). The circle drawn in render() shrinks as
-        # the target ages, but the clickable area never does, so a
-        # late click well outside the small visible circle can still
-        # register as a hit. See Task 1 in the README.
+        return math.hypot(self.x - x, self.y - y) <= self.base_radius
+        # Hit-testing uses the same radius that render() draws, so the
+        # clickable area always matches the visible (shrinking) circle.
+        return math.hypot(self.x - x, self.y - y) <= self.visual_radius()
```

Task 2: Game over condition

You: finish task 2 and commit

Claude: Replaced the console print with an on-screen Game Over overlay showing final score, accuracy, hits and misses. Input is ignored for about half a second so a last frantic click doesn't dismiss it; afterwards any key or click exits cleanly (`should_quit` checked in `main.py`). Commit a452959.

```
@@ -25,4 +25,7 @@ class GameEngine:
    self.font = pygame.font.SysFont("Arial", 26)
    self.game_over = False
+    self.should_quit = False
+    self.game_over_frames = 0 # frames since the round ended
+    self.big_font = pygame.font.SysFont("Arial", 56, bold=True)

    def _spawn_target(self):
```

```

@@ -33,4 +36,10 @@ class GameEngine:
    def handle_event(self, event):
        if self.game_over:
+           # Ignore input for a moment so a frantic last click doesn't
+           # instantly dismiss the screen before it can be read.
+           if self.game_over_frames > 30 and event.type in (
+               pygame.MOUSEBUTTONDOWN, pygame.KEYDOWN
+           ):
+               self.should_quit = True
+               return
        if event.type == pygame.MOUSEBUTTONDOWN:
@@ -53,4 +62,5 @@ class GameEngine:
    def update(self):
        if self.game_over:
+           self.game_over_frames += 1
        return

@@ -72,8 +82,14 @@ class GameEngine:

    def render(self, screen):
+       if self.game_over:
+           self._render_game_over(screen)
+           return

+       r = int(self.target.visual_radius())
+       pygame.draw.circle(screen, RED, (self.target.x, self.target.y), r)
+       pygame.draw.circle(screen, WHITE, (self.target.x, self.target.y), r, 2)
+       self._render_hud(screen)

+   def _render_hud(self, screen):
+       score_text = self.font.render(f"Score: {self.score}", True, WHITE)
+       screen.blit(score_text, (10, 10))
@@ -86,6 +102,23 @@ class GameEngine:
    screen.blit(acc_text, (self.width // 2 - 90, 10))

-       if self.game_over and not getattr(self, "_game_over_logged", False):
-           # NOTE: no proper game-over screen yet - see Task 2 in the README.
-           print(f"Time's up! Final score: {self.score} Accuracy: {self.accuracy()}%")
-           self._game_over_logged = True
+   def _blit_centered(self, screen, surf, y):
+       screen.blit(surf, (self.width // 2 - surf.get_width() // 2, y))
+
+   def _render_game_over(self, screen):
+       self._render_hud(screen)
+       overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
+       overlay.fill((0, 0, 0, 180))
+       screen.blit(overlay, (0, 0))
+
+       self._blit_centered(screen, self.big_font.render("GAME OVER", True, WHITE), 110)
+       self._blit_centered(screen, self.font.render(f"Final Score: {self.score}", True, WHITE), 200)
+       self._blit_centered(screen, self.font.render(f"Accuracy: {self.accuracy()}%", True, WHITE), 240)
+       self._blit_centered(
+           screen,
+           self.font.render(f"Hits: {self.hits} Misses: {self.misses}", True, WHITE),
+           280,
+       )
+       if self.game_over_frames > 30:
+           self._blit_centered(
+               screen, self.font.render("Press any key or click to exit", True, (200, 200, 200)), 370
+           )
@@ -32,4 +32,6 @@ def main():
    engine.update()
    engine.render(SCREEN)
+   if engine.should_quit:
+       running = False

    pygame.display.flip()

```

Task 3: Replay option with difficulty

You: similarly task 3

Claude: Added a DIFFICULTIES table (Easy: radius 50 to 18, 2 s lifespan; Medium: 40 to 12, 1.5 s; Hard: 28 to 8, 1 s) and a start_round(difficulty) method that resets score, hits, misses, timer and target. The Game Over screen now shows Easy / Medium / Hard buttons (keys 1/2/3 or click) and a Quit button (Q, Esc or click). Tested that each choice starts a fresh round with the right target size and speed, and that Quit exits. Commit 9da1c6e.

```
@@ -5,4 +5,14 @@ from .target import Target
# Game Engine

+## Difficulty presets: (base_radius, min_radius, lifespan_frames at 60 FPS)
+DIFFICULTIES = {
+    "Easy": {"base_radius": 50, "min_radius": 18, "lifespan_frames": 120},
+    "Medium": {"base_radius": 40, "min_radius": 12, "lifespan_frames": 90},
+    "Hard": {"base_radius": 28, "min_radius": 8, "lifespan_frames": 60},
+}
+GRAY = (200, 200, 200)
+BUTTON = (70, 70, 85)
+BUTTON_HOVER = (100, 100, 125)
+
+WHITE = (255, 255, 255)
+RED = (220, 60, 60)
@@ -15,35 +25,63 @@ class GameEngine:
    self.margin = 60
    self.hud_height = 60
-    self.target = self._spawn_target()
-
-    self.round_seconds = 30
+    self.font = pygame.font.SysFont("Arial", 26)
+    self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
+    self.should_quit = False
+    self.difficulty = "Medium"
+
+    # Game-over menu buttons: three difficulties in a row + quit
+    bw, bh, gap = 150, 50, 20
+    x0 = self.width // 2 - (3 * bw + 2 * gap) // 2
+    self.buttons = {
+        name: pygame.Rect(x0 + i * (bw + gap), 340, bw, bh)
+        for i, name in enumerate(DIFFICULTIES)
+    }
+    self.quit_button = pygame.Rect(self.width // 2 - 75, 410, 150, 44)
+
+    self.start_round(self.difficulty)
+
+    def start_round(self, difficulty):
+        """(Re)start a round with the chosen difficulty."""
+        self.difficulty = difficulty
+        self.time_left_frames = self.round_seconds * 60
+
+        self.hits = 0
+        self.misses = 0
+        self.score = 0
+        self.font = pygame.font.SysFont("Arial", 26)
+        self.game_over = False
+        self.should_quit = False
+        self.game_over_frames = 0 # frames since the round ended
+        self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
+        self.target = self._spawn_target()
+
+    def _spawn_target(self):
+        x = random.randint(self.margin, self.width - self.margin)
+        y = random.randint(self.margin + self.hud_height, self.height - self.margin)
+        return Target(x, y)
+        return Target(x, y, **DIFFICULTIES[self.difficulty])
+
+    def handle_event(self, event):
+        if self.game_over:
+            # Ignore input for a moment so a frantic last click doesn't
+            # instantly dismiss the screen before it can be read.
+            if self.game_over_frames > 30 and event.type in (
+                pygame.MOUSEBUTTONDOWN, pygame.KEYDOWN
+            ):
+                self.should_quit = True
+                self._handle_game_over_event(event)
+                return
+            if event.type == pygame.MOUSEBUTTONDOWN:
+                self._handle_click(event.pos)
+
+    def _handle_game_over_event(self, event):
+        # Ignore input for a moment so a frantic last click doesn't
+        # instantly pick an option before the screen can be read.
+        if self.game_over_frames <= 30:
```

```

+         return
+     if event.type == pygame.KEYDOWN:
+         keys = {pygame.K_1: "Easy", pygame.K_2: "Medium", pygame.K_3: "Hard"}
+         if event.key in keys:
+             self.start_round(keys[event.key])
+         elif event.key in (pygame.K_ESCAPE, pygame.K_q):
+             self.should_quit = True
+     elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+         for name, rect in self.buttons.items():
+             if rect.collidepoint(event.pos):
+                 self.start_round(name)
+                 return
+         if self.quit_button.collidepoint(event.pos):
+             self.should_quit = True
+
+     def _handle_click(self, pos):
+         x, y = pos
@@ -111,14 +149,23 @@ class GameEngine:
    screen.blit(overlay, (0, 0))

-     self._blit_centered(screen, self.big_font.render("GAME OVER", True, WHITE), 110)
-     self._blit_centered(screen, self.font.render(f"Final Score: {self.score}", True, WHITE), 200)
-     self._blit_centered(screen, self.font.render(f"Accuracy: {self.accuracy()}%", True, WHITE), 240)
+     self._blit_centered(screen, self.big_font.render("GAME OVER", True, WHITE), 70)
+     self._blit_centered(screen, self.font.render(f"Final Score: {self.score}", True, WHITE), 160)
+     self._blit_centered(screen, self.font.render(f"Accuracy: {self.accuracy()}%", True, WHITE), 200)
+     self._blit_centered(
+         screen,
+         self.font.render(f"Hits: {self.hits}    Misses: {self.misses}", True, WHITE),
+         280,
+         240,
+     )
+     if self.game_over_frames > 30:
+         self._blit_centered(
+             screen, self.font.render("Press any key or click to exit", True, (200, 200, 200)), 370
+         )
+         self._blit_centered(screen, self.font.render("Play again - choose difficulty:", True, GRAY), 300)
+         mouse = pygame.mouse.get_pos()
+         for i, (name, rect) in enumerate(self.buttons.items(), start=1):
+             self._draw_button(screen, rect, f"{i}. {name}", mouse)
+             self._draw_button(screen, self.quit_button, "Q. Quit", mouse)
+
+     def _draw_button(self, screen, rect, label, mouse):
+         color = BUTTON_HOVER if rect.collidepoint(mouse) else BUTTON
+         pygame.draw.rect(screen, color, rect, border_radius=8)
+         pygame.draw.rect(screen, WHITE, rect, 2, border_radius=8)
+         text = self.font.render(label, True, WHITE)
+         screen.blit(text, text.get_rect(center=rect.center))

```

Task 4: Sound feedback

You: now task 4

Claude: Added game/sound.py with a SoundManager that generates three short tones in code (no audio files): a rising blip for a hit, a low buzz for a miss or timed-out target, and a long descending tone when the round ends. It disables itself if no audio device is available. Wired into hit, miss, timeout and round-end in the engine. Tested that the right sound triggers for each event. Commit e0e9d7b; also a small commit adding a .gitignore for __pycache__.

```
@@ -2,4 +2,5 @@ import pygame
import random
from .target import Target
+from .sound import SoundManager

# Game Engine
@@ -28,4 +29,5 @@ class GameEngine:
    self.font = pygame.font.SysFont("Arial", 26)
    self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
+    self.sounds = SoundManager()
    self.should_quit = False
    self.difficulty = "Medium"
@@ -89,7 +91,9 @@ class GameEngine:
    self.hits += 1
    self.score += 1
+    self.sounds.play_hit()
    self.target = self._spawn_target()
    else:
        self.misses += 1
+        self.sounds.play_miss()

    def handle_input(self):
@@ -106,4 +110,5 @@ class GameEngine:
    if self.time_left_frames <= 0:
        self.game_over = True
        self.sounds.play_end()
    return

@@ -111,4 +116,5 @@ class GameEngine:
    if self.target.expired():
        self.misses += 1 # Letting a target time out counts as a miss too
+        self.sounds.play_miss()
    self.target = self._spawn_target()

new file mode 100644
@@ -0,0 +1,49 @@
+import math
+import struct
+
+import pygame
+
+SAMPLE_RATE = 44100
+
+def _tone(freq_start, freq_end, duration, volume=0.4):
+    """Build a short sine sweep as a pygame Sound (no audio files needed)."""
+    n = int(SAMPLE_RATE * duration)
+    samples = []
+    phase = 0.0
+    for i in range(n):
+        t = i / n
+        freq = freq_start + (freq_end - freq_start) * t
+        phase += 2 * math.pi * freq / SAMPLE_RATE
+        envelope = min(1.0, (1 - t) * 4) # quick fade-out to avoid clicks
+        samples.append(int(32767 * volume * envelope * math.sin(phase)))
+    data = struct.pack("<" + "h" * n, *samples)
+    return pygame.mixer.Sound(buffer=data)
+
+class SoundManager:
+    """Hit / miss / round-end sounds. Silently disabled if audio is unavailable."""
+
+    def __init__(self):
+        self.enabled = False
+        try:
+            pygame.mixer.init(frequency=SAMPLE_RATE, size=-16, channels=1)
+            self.hit = _tone(660, 990, 0.12) # rising blip
+            self.miss = _tone(220, 110, 0.18) # low falling buzz
+            self.end = _tone(880, 220, 0.6) # long descending tone
+            self.enabled = True
+        except pygame.error:
+            pass
+
+    def _play(self, sound):
+        if self.enabled:
```

```

+         sound.play()
+
+     def play_hit(self):
+         self._play(self.hit)
+
+     def play_miss(self):
+         self._play(self.miss)
+
+     def play_end(self):
+         self._play(self.end)

```

Final: after video and this document

You: i recorded the after video now now give me the chat as a pdf ... put this link also in the pdf

Claude: Generated this PDF with the share link included.

Commit history

Commit	Message
9eb8ee3	Task 1: hit-test against the visible (shrunk) radius
a452959	Task 2: game over screen showing final score and accuracy, waits for input
9da1c6e	Task 3: replay option with Easy/Medium/Hard difficulty selection or quit
e0e9d7b	Task 4: sound effects for hit, miss/timeout, and round end
569085b	Chore: add .gitignore for __pycache__